

10 Hermes Agent 安装与使用教程

基于 Hermes Agent 官方文档（v0.8.0）撰写的安装与使用指南。官网：<https://hermes-agent.nousresearch.com/docs/zh-Hans>

一、安装

1.1 环境要求

依赖	最低版本
Python	3.11+
Node.js	22+（部分功能需要）
Git	任意（安装器可自动处理可移植版）
操作系统	Linux / macOS / WSL2 / Windows / Android (Termux)

1.2 安装方式

Linux / macOS / WSL2 / Termux（推荐 —— 跟踪 main 分支）

```
curl -fsSL https://raw.githubusercontent.com/NousResearch/hermes-agent/main/scripts/install.sh | bash
```

安装完成后运行：

```
source ~/.bashrc
```

Windows（PowerShell，早期 Beta）

```
iex (irm https://raw.githubusercontent.com/NousResearch/hermes-agent/main/scripts/install.ps1)
```

安装程序自动处理：uv、Python 3.11、Node.js 22、ripgrep、ffmpeg 以及可移植 Git。仓库克隆到 %LOCALAPPDATA%\hermes\hermes-agent，创建虚拟环境，将 hermes 添加到用户 PATH。

注意： Windows 原生支持除浏览器仪表板聊天面板外的所有功能（仪表板仅限 WSL2，因使用 POSIX PTY）。

pip 安装（跟踪带标签的发布版本）

```
pip install hermes-agent
hermes postinstall
```

Docker 安装

Hermes 提供官方 Docker 镜像 [nousresearch/hermes-agent](#)。镜像本身无状态，所有用户数据存储在宿主机挂载的 `/opt/data` 目录中。拉取新版本即可升级，不会丢失任何配置。

镜像基于 **debian:13.4**，内置 Python 3 + 全部依赖、Node.js + npm、Playwright + Chromium、ripgrep、ffmpeg、git、docker-cli、openssh-client、WhatsApp 桥接，以 s6-overlay v3 作为 PID 1 进程监管器。

注意： Docker 与 Hermes 有两种交集方式：

1. **在 Docker 中运行 Hermes**（本节的内容） — Agent 本身在容器内运行
2. **Docker 作为终端后端** — Agent 在宿主机运行，命令在持久化 Docker 沙箱容器中执行（参见配置参考中的 `terminal.backend: docker`）

快速开始 —— 首次配置：

```
# 创建数据目录
mkdir -p ~/.hermes

# 交互式运行设置向导（只需执行一次）
docker run -it --rm \
  -v ~/.hermes:/opt/data \
  nousresearch/hermes-agent setup
```

向导会提示输入 API 密钥并写入 `~/.hermes/.env`。

以 Gateway 模式运行：

```
docker run -d \
  --name hermes \
  --restart unless-stopped \
  -v ~/.hermes:/opt/data \
  -p 8642:8642 \
  nousresearch/hermes-agent gateway run
```

端口 8642 暴露 Gateway 的 OpenAI 兼容 API 和健康检查端点。仅使用聊天平台（Telegram、Discord 等）时端口可选。

启用 API Server：

```
docker run -d \
  --name hermes \
```

```
--restart unless-stopped \  
-v ~/.hermes:/opt/data \  
-p 8642:8642 \  
-e API_SERVER_ENABLED=true \  
-e API_SERVER_HOST=0.0.0.0 \  
-e API_SERVER_KEY="$(openssl rand -hex 32)" \  
-e API_SERVER_CORS_ORIGINS='*' \  
nousresearch/hermes-agent gateway run
```

安全警告： 在面向互联网的机器上开放端口存在安全风险，请确保了解相关风险。

运行 Dashboard：

```
docker run -d \  
  --name hermes \  
  --restart unless-stopped \  
  -v ~/.hermes:/opt/data \  
  -p 8642:8642 \  
  -e HERMES_DASHBOARD=1 \  
  nousresearch/hermes-agent gateway run
```

Dashboard 环境变量：

环境变量	描述	默认值
HERMES_DASHBOARD	设为 1 以启动 dashboard	未设置
HERMES_DASHBOARD_HOST	Dashboard HTTP 绑定地址	127.0.0.1
HERMES_DASHBOARD_PORT	Dashboard HTTP 端口	9119
HERMES_DASHBOARD_TUI	设为 1 启用浏览器内 Chat 标签页（PTY/WebSocket）	未设置
HERMES_DASHBOARD_INSECURE	设为 1 禁用 OAuth 鉴权门控	未设置

Dashboard 默认绑定回环地址。发布到网络时设置 `HERMES_DASHBOARD_HOST=0.0.0.0`。绑定非回环地址 + 注册了 `DashboardAuthProvider` 时，OAuth 鉴权自动启用。未注册 provider 且绑定非回环地址时，dashboard 启动失败关闭并给出具体错误。

备注： Dashboard 在容器内作为受监管的 s6 服务运行。崩溃后 s6-overlay 自动重启，无需重启容器。不支持将 dashboard 作为独立容器运行——其 gateway 存活检测需要与 gateway 共享 PID 命名空间。

交互式运行（CLI 聊天）：

```
# 对已有数据目录打开交互式聊天  
docker run -it --rm \  
  -v ~/.hermes:/opt/data \  
  nousresearch/hermes-agent
```

```
# 或在已运行容器内直接执行
docker exec -it hermes /opt/hermes/.venv/bin/hermes
```

持久化卷 —— /opt/data:

路径	内容
.env	API 密钥和机密
config.yaml	所有 Hermes 配置
SOUL.md	Agent 个性/身份
sessions/	对话历史
memories/	持久化记忆存储
skills/	已安装的技能
cron/	定时任务定义
hooks/	事件 hook
logs/	运行时日志
skins/	自定义 CLI 皮肤

注意：切勿同时对同一数据目录运行两个 Hermes gateway 容器——会话文件和记忆存储不支持并发写入。

多 Profile 支持：

在 Docker 下推荐每个 profile 一个容器，各自绑定挂载独立目录：

```
# 工作 profile
docker run -d \
  --name hermes-work \
  --restart unless-stopped \
  -v ~/.hermes-work:/opt/data \
  -p 8642:8642 \
  nousresearch/hermes-agent gateway run

# 个人 profile
docker run -d \
  --name hermes-personal \
  --restart unless-stopped \
  -v ~/.hermes-personal:/opt/data \
  -p 8643:8642 \
  nousresearch/hermes-agent gateway run
```

独立容器的优势：隔离性强（崩溃不影响其他 profile）、独立生命周期（可分别升级/重启）、清晰的端口和网络隔离、避免并发写入风险。

Docker Compose 示例:

```
services:
  hermes:
    image: nousresearch/hermes-agent:latest
    container_name: hermes
    restart: unless-stopped
    command: gateway run
    ports:
      - "8642:8642" # gateway API
      - "9119:9119" # dashboard (仅 HERMES_DASHBOARD=1 时生效)
    volumes:
      - ~/.hermes:/opt/data
    environment:
      - HERMES_DASHBOARD=1
      # 取消注释以直接转发环境变量而非使用 .env 文件:
      # - ANTHROPIC_API_KEY=${ANTHROPIC_API_KEY}
      # - OPENAI_API_KEY=${OPENAI_API_KEY}
      # - TELEGRAM_BOT_TOKEN=${TELEGRAM_BOT_TOKEN}
    deploy:
      resources:
        limits:
          memory: 4G
          cpus: "2.0"
```

多 profile Docker Compose:

```
services:
  hermes-work:
    image: nousresearch/hermes-agent:latest
    container_name: hermes-work
    restart: unless-stopped
    command: gateway run
    ports:
      - "8642:8642"
    volumes:
      - ~/.hermes-work:/opt/data

  hermes-personal:
    image: nousresearch/hermes-agent:latest
    container_name: hermes-personal
    restart: unless-stopped
    command: gateway run
    ports:
      - "8643:8642"
    volumes:
      - ~/.hermes-personal:/opt/data
```

启动:

```
docker compose up -d
docker compose logs -f
```

环境变量转发：

API 密钥从 `/opt/data/.env` 读取，也可直接传递（覆盖 `.env` 中的值）：

```
docker run -it --rm \
-v ~/.hermes:/opt/data \
-e ANTHROPIC_API_KEY="sk-ant-..." \
-e OPENAI_API_KEY="sk-..." \
nousresearch/hermes-agent
```

适用于不希望密钥写入磁盘的 CI/CD 或密钥管理器集成场景。

资源限制：

资源	最低	推荐
内存	1 GB	2–4 GB
CPU	1 核	2 核
磁盘（数据卷）	500 MB	2+ GB

浏览器自动化（Playwright/Chromium）最耗内存。如不需要浏览器工具，1 GB 即可。

```
docker run -d \
--name hermes \
--restart unless-stopped \
--memory=4g --cpus=2 \
-v ~/.hermes:/opt/data \
nousresearch/hermes-agent gateway run
```

在容器中安装更多工具：

方式	适用场景
<code>npx / uvx</code>	npm 或 PyPI 工具，按需获取，配置写入 <code>/opt/data</code>
<code>apt-get install</code> + 记住命令	其他工具，安装后在容器生命周期内可用，重启后重新安装
构建派生镜像	每次启动必须立即可用的工具，写入 Dockerfile 层
Sidecar 容器	自带服务的复杂工具（数据库、队列等），独立容器通过共享网络访问
提交 issue/PR	对大多数用户有用的工具，贡献到上游

连接本地推理服务器（vLLM、Ollama 等）：

Docker Compose（推荐，最可靠）：

```
services:
  vllm:
    image: vllm/vllm-openai:latest
    container_name: vllm
    command: >
      --model Qwen/Qwen2.5-7B-Instruct
      --served-model-name my-model
      --host 0.0.0.0 --port 8000
    ports:
      - "8000:8000"
    networks:
      - hermes-net
    deploy:
      resources:
        reservations:
          devices:
            - capabilities: [gpu]

  hermes:
    image: nousresearch/hermes-agent:latest
    container_name: hermes
    restart: unless-stopped
    command: gateway run
    ports:
      - "8642:8642"
    volumes:
      - ~/.hermes:/opt/data
    networks:
      - hermes-net

networks:
  hermes-net:
    driver: bridge
```

然后在 `config.yaml` 中使用容器名称作为主机名：

```
model:
  provider: custom
  model: my-model
  base_url: http://vllm:8000/v1
  api_key: "none"
```

关键点： 使用容器名称（如 `vllm`）作为主机名——而非 `localhost` 或 `127.0.0.1`，它们指向 Hermes 容器自身。`model` 值须与 `--served-model-name` 一致。`base_url` 末尾不加斜杠。

宿主机推理服务器（非 Docker）：

```
# macOS / Windows - 使用 host.docker.internal
docker run -d --name hermes -v ~/.hermes:/opt/data -p 8642:8642 \
  nousresearch/hermes-agent gateway run

# config.yaml → base_url: http://host.docker.internal:8000/v1

# Linux - 使用 --network host
docker run -d --name hermes --network host -v ~/.hermes:/opt/data \
  nousresearch/hermes-agent gateway run

# config.yaml → base_url: http://127.0.0.1:8000/v1
```

Ollama 同理，使用 `host.docker.internal:11434` (macOS/Windows) 或 `127.0.0.1:11434` (Linux `--network host`)。

验证连通性：

```
docker exec hermes curl -s http://vllm:8000/v1/models
```

升级：

```
# 纯 Docker
docker pull nousresearch/hermes-agent:latest
docker rm -f hermes
docker run -d --name hermes --restart unless-stopped \
  -v ~/.hermes:/opt/data \
  nousresearch/hermes-agent gateway run

# Docker Compose
docker compose pull
docker compose up -d
```

故障排查：

问题	解决
容器立即退出	<code>docker logs hermes</code> ，常见原因： <code>.env</code> 缺失或无效、端口冲突
Permission denied	<code>chmod -R 755 ~/.hermes</code> 或设置 <code>HERMES_UID/HERMES_GID</code> 匹配宿主机用户
浏览器工具无法使用	添加 <code>--shm-size=1g</code> (Playwright 需要共享内存)
Gateway 无法重连	<code>docker restart hermes</code> (<code>--restart unless-stopped</code> 处理大多数瞬时故障)
检查健康状态	<code>docker logs --tail 50 hermes</code> 、 <code>docker stats hermes</code>

权限模型： s6-overlay 的 `/init` 以 root 运行，在首次启动时对卷执行 `chown`，然后通过 `s6-setuidgid` 将每个受监管服务及主程序降权至 `hermes` 用户。以 root 启动 `hermes gateway run` 默认被拒绝（可能在

`/opt/data` 中留下 root 所有的文件)。仅在有意接受该风险时设置 `HERMES_ALLOW_ROOT_GATEWAY=1`。

1.3 安装目录结构

安装方式	代码位置	二进制文件	数据目录
pip install	Python site-packages	<code>~/.local/bin/hermes</code>	<code>~/.hermes/</code>
用户级 git	<code>~/.hermes/hermes-agent/</code>	<code>~/.local/bin/hermes</code> （符号链接）	<code>~/.hermes/</code>
根模式	<code>/usr/local/lib/hermes-agent/</code>	<code>/usr/local/bin/hermes</code>	<code>/root/.hermes/</code> 或 <code>\$HERMES_HOME</code>

1.4 验证安装

```
hermes --version
hermes doctor
```

`hermes doctor` 会检查所有依赖、配置完整性，以及安全公告（如供应链扫描）。

二、快速开始

2.1 配置 LLM Provider

安装完成后首次运行 `hermes` 会进入交互式配置向导。你也可以直接编辑配置文件：

```
# ~/.hermes/config.yaml
api:
  provider: openrouter      # 或 anthropic / openai / nous
  model: anthropic/claude-sonnet-4-6
  base_url: https://openrouter.ai/api/v1
  api_key: ${OPENROUTER_API_KEY}  # 支持环境变量引用
```

支持 15+ Provider 的开箱即用配置，包括 OpenAI、Anthropic、OpenRouter、Nous Portal、DeepSeek、Qwen 等。

2.2 开始对话

```
# 经典 CLI（基于 prompt_toolkit，多行编辑、斜杠命令补全）
hermes

# 现代 TUI（模态覆盖层、鼠标选择）
hermes --tui
```

```
# 单次查询（非交互模式）
hermes chat -q "解释这段代码：cat main.py"

# 恢复最近的会话
hermes --continue
# 或
hermes -c
```

三、CLI 使用详解

3.1 状态栏

输入区上方实时显示：

- **模型名称**（当前使用的 LLM）
- **令牌计数**（已用 / 最大）
- **上下文进度条**：绿色 <50%，黄色 50-80%，橙色 80-95%，红色 ≥95%
- **估计成本**（美元）
- **压缩次数 [N]**（当前会话中自动压缩次数）
- **后台任务数**
- **会话时长**
- **YOLO 模式警告**

3.2 键盘快捷键

快捷键	功能
Enter	发送消息
Alt+Enter / Ctrl+J / Shift+Enter	换行
Ctrl+B	切换语音录制
Ctrl+G	在 \$EDITOR 中打开输入缓冲区
Ctrl+C	中断 Agent（2 秒内双击强制退出）
Ctrl+D	退出
Ctrl+Z	将 Hermes 置于后台（Unix）

3.3 核心斜杠命令

命令	功能
/help	显示帮助
/model	切换当前模型
/tools	查看可用工具

命令	功能
<code>/skills browse</code>	浏览已安装技能
<code>/background <prompt></code>	在后台运行独立 Agent 任务
<code>/skin</code>	更换主题
<code>/voice on</code>	开启语音输入
<code>/voice off</code>	关闭语音输入
<code>/reasoning high</code>	开启高推理模式
<code>/reasoning off</code>	关闭推理模式
<code>/title <名称></code>	为当前会话命名
<code>/compress</code>	手动触发上下文压缩
<code>/yolo</code>	切换 YOLO 模式（跳过命令审批）
<code>/busy</code>	设置忙时输入模式

3.4 忙时输入模式

通过 `/busy` 或配置 `display.busy_input_mode` 设置：

模式	行为
<code>interrupt</code> （默认）	立即中断正在进行的操作
<code>queue</code>	静默排队，待 Agent 完成后发送
<code>steer</code>	通过 <code>/steer</code> 注入到当前运行中（不中断）

3.5 后台会话

在独立守护线程中运行 Agent 任务，完全非阻塞：

```
# 启动后台任务
/background 帮我研究一下这个项目里所有的 API 端点

# 启动时预加载技能
hermes -s skill1,skill2
```

支持多个并发后台任务，每个有独立 ID。

3.6 隔离 Worktree 模式

在隔离的 git worktree 中运行 Agent：

```
hermes -w
```

四、高级功能

4.1 内存系统

自动记忆： Agent 在对话中自动通过 `memory` 工具管理 MEMORY.md 和 USER.md。

手动管理：

```
# 查看当前记忆
cat ~/.hermes/memories/MEMORY.md
cat ~/.hermes/memories/USER.md

# 对话中操作记忆
# Agent 会自行调用 memory 工具，也可以直接要求：
"把"这个项目用的是 Go 1.22 和 PostgreSQL"记到记忆里"
```

配置外部 Memory Provider：

```
# 列出可用的记忆后端
hermes memory list

# 配置 Honcho 作为外部后端
hermes memory setup honcho
```

4.2 技能系统 (Skills)

Agent 可以在复杂任务成功后**自动创建技能**，把方法论固化下来。

技能目录结构：

```
~/.hermes/skills/
├── my-skill/
│   ├── SKILL.md           # YAML frontmatter + Markdown 正文
│   ├── references/        # 参考资料
│   ├── templates/         # 模板文件
│   ├── scripts/           # 脚本
│   └── assets/            # 其他资源
```

4.3 会话管理

```
# 查看会话列表
hermes sessions list

# 恢复指定会话
hermes sessions resume <session_id>

# 搜索历史对话
# 在对话中使用 session_search 工具
```

所有会话存储在 SQLite `~/.hermes/state.db` 中，带 FTS5 全文搜索。

4.4 Gateway 模式（多平台消息网关）

Hermes 可以作为消息网关运行，同时服务 11 个平台：

```
hermes gateway
```

支持平台：Telegram、Discord、Slack、WhatsApp、Signal、Matrix、Email、HomeAssistant、Mattermost、DingTalk（钉钉）、Feishu（飞书）。

Gateway 配置见 `~/.hermes/config.yaml` 的 `gateway` 部分。

4.5 安全模式

```
# YOLO 模式（跳过命令审批，有红色横幅警告）
hermes --yolo
# 或在对话中输入 /yolo

# 通过环境变量启用
HERMES_YOLO_MODE=1 hermes

# Docker 沙箱模式（推荐生产环境使用）
# 在 config.yaml 中设置：
# terminal:
#   backend: docker
```

五、配置参考

5.1 核心配置项

```
# ~/.hermes/config.yaml

# Agent 配置
agent:
```

```

max_turns: 90 # 最大迭代次数
tool_delay: 1.0 # 工具调用延迟

# 压缩配置
compression:
  threshold: 0.50 # 触发阈值 (50% 上下文窗口)
  protect_last_n: 20 # 尾部保护消息数
  target_ratio: 0.20 # 摘要目标比例
  summary_model: "" # 摘要专用模型 (留空用主模型)

# 终端配置
terminal:
  backend: local # local / docker / ssh / modal / daytona
/ singularity
  env_passthrough: [] # 传递的环境变量
  docker:
    memory_limit: 5GB
    disk_limit: 50GB

# 安全配置
security:
  approvals:
    mode: manual # manual / smart / off
  allow_private_urls: false # 允许访问私有网络
  allow_lazy_installs: true # 允许惰性安装依赖

# 记忆配置
memory:
  provider: "" # 外部 provider, 留空只用内置

```

5.2 凭证池配置

支持同一 Provider 使用多个 API Key，调度策略可选：

```

api:
  provider: openrouter
  credentials:
    - key: ${KEY_1}
    - key: ${KEY_2}
  credential_strategy: round_robin # fill_first / round_robin / random /
least_used

```

5.3 Fallback Provider 链

```

api:
  provider: anthropic
  model: claude-sonnet-4-6
  fallback_providers:
    - provider: openrouter

```

```
model: anthropic/claude-sonnet-4-6  
- provider: openai  
model: gpt-5
```

六、批量训练数据生成

6.1 准备数据集

```
{"prompt": "用 Python 写一个快速排序算法"}  
{"prompt": "创建一个 Flask REST API，包含用户 CRUD 操作"}  
{"prompt": "用 React 写一个 Todo 列表组件"}
```

6.2 运行批量生成

```
hermes batch-run --dataset dataset.jsonl --output trajectories.jsonl --workers 4
```

6.3 压缩训练轨迹

```
hermes compress-trajectories --input trajectories.jsonl --output compressed.jsonl  
--target-tokens 15250
```

七、常见问题

如何查看日志？

```
# 实时查看  
tail -f ~/.hermes/logs/hermes.log  
  
# 或通过 doctor 检查  
hermes doctor
```

如何更新？

```
# git 安装
hermes update

# pip 安装
pip install --upgrade hermes-agent
hermes postinstall
```

上下文窗口不足怎么办？

Hermes 会自动触发压缩（上下文字符数 > 50% 窗口时）。也可以手动触发：

```
/compress
```

如何在 Docker 中运行？

两种方式：

1. Hermes 本身运行在 Docker 容器中（推荐生产环境）：

参见上方 [Docker 安装](#) 章节。

```
docker run -d --name hermes --restart unless-stopped \
-v ~/.hermes:/opt/data -p 8642:8642 \
nousresearch/hermes-agent gateway run
```

2. Docker 作为终端沙箱后端（Agent 在宿主机，命令在容器中执行）：

```
# 在 config.yaml 中设置：
# terminal:
#   backend: docker
```

Windows 下的限制？

Windows 原生支持所有核心功能。基于浏览器的仪表板聊天面板仅限 WSL2（依赖 POSIX PTY）。

八、完整命令行参考

```
# 基础
hermes
hermes --tui
hermes chat -q "问题"
hermes -c

# 启动经典 CLI
# 启动现代 TUI
# 单次查询
# 恢复最近会话
```

```
hermes -w                # Worktree 隔离模式
hermes -s skill1,skill2  # 预加载技能

# 安全
hermes --yolo            # YOLO 模式

# 会话
hermes sessions list     # 列出会话
hermes sessions resume <id> # 恢复会话

# 记忆
hermes memory list       # 列出记忆后端
hermes memory setup <name> # 配置外部后端

# 网关
hermes gateway           # 启动消息网关

# 批量
hermes batch-run ...     # 批量生成训练数据

# 维护
hermes doctor            # 系统诊断
hermes update            # 更新
hermes postinstall       # 安装后配置
```

更多信息请查看 [Hermes Agent 官网](#)